FEA HW4

Ben Sarfati 941180069

September 12 2023

1

1.1

To derive a finite element formulation for the problem we first define the residual in the domain, introducing an approximate form of the solution $\tilde{T} = \phi_j(x)a_j(t)$:

$$R = -\frac{\partial T}{\partial t} + \frac{\partial}{\partial x}\left(\kappa(x)\frac{\partial T}{\partial x}\right) = -\phi_j(x)\frac{\partial a_j(t)}{\partial t} + \frac{\partial}{\partial x}\left(\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right)a_j(t)$$

Now we demand

$$\int_0^L \phi_i(x)R dx = 0$$

$$\Rightarrow \int_0^L \phi_i(x)\Big[-\phi_j(x)\frac{\partial a_j(t)}{\partial t} + \frac{\partial}{\partial x}\left(\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right)a_j(t)\Big]dx = 0$$

$$\Rightarrow \frac{\partial a_j(t)}{\partial t}\int_0^L \phi_i(x)\phi_j(x)dx - \int_0^L \phi_i(x)\frac{\partial}{\partial x}\left(\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right)a_j(t)dx = 0$$

$$\phi_i(x)\frac{\partial}{\partial x}\left(\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right)a_j(t) = \frac{\partial}{\partial x}\left(\phi_i(x)\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right)a_j(t) - \frac{\partial \phi_i(x)}{\partial x}\kappa(x)\frac{\partial \phi_j(x)}{\partial x}a_j(t)$$

$$\Rightarrow \frac{\partial a_j(t)}{\partial t}\int_0^L \phi_i(x)\phi_j(x)dx + \int_0^L \frac{\partial \phi_i(x)}{\partial x}\kappa(x)\frac{\partial \phi_j(x)}{\partial x}a_j(t)dx - \left[\phi_i(x)\kappa(x)\frac{\partial \phi_j(x)}{\partial x}\right]_0^L a_j(t) = 0$$

For N shape functions, we demand $i \neq 1, i \neq N \Rightarrow \phi_i(0) = \phi_i(L) = 0$ in order to get rid of the dirichlet boundary terms $\forall 1 < i < N$. We are left with

$$i \neq 1, i \neq N \Rightarrow \frac{\partial a_j(t)}{\partial t}\int_0^L \phi_i(x)\phi_j(x)dx + \int_0^L \frac{\partial \phi_i(x)}{\partial x}\kappa(x)\frac{\partial \phi_j(x)}{\partial x}a_j(t)dx = 0$$

This is the weak formulation of the problem. To divide the domain $L = 1$ into $n = 4$ linear elements we introduce $N = n + 1 = 5$ nodal points x_j, and the following transformation $\forall e \leq n \in \mathbb{N}$ (e is not in Einstein notation):

$$x(\xi^e) = \frac{h^e}{2}\xi^e + \frac{x_e + x_{e+1}}{2}$$

Where

$$h^e = \frac{L}{n} = \frac{1}{4}$$

$$x_j = (j-1)h^e = (j-1)\frac{L}{n} = \frac{j-1}{4}$$

$$J^e = \frac{dx}{d\xi^e} = \frac{h^e}{2} = \frac{1}{8}$$

For each element there are only two shape functions so we will define

$$\begin{cases} \phi_1^e(x) = \phi_e(x) \\ \\ \phi_2^e(x) = \phi_{e+1}(x) \end{cases}$$

$$\begin{cases} \hat{\phi}_1(\xi^e) = \frac{1}{2}(1-\xi^e) \\ \\ \hat{\phi}_2(\xi^e) = \frac{1}{2}(1+\xi^e) \end{cases}$$

$$\begin{cases} B_1(\xi^e) = \frac{d\hat{\phi}_1(\xi^e)}{d\xi^e} = -\frac{1}{2} \\ \\ B_2(\xi^e) = \frac{d\hat{\phi}_2(\xi^e)}{d\xi^e} = \frac{1}{2} \end{cases}$$

Note that

$$\phi_1^e(x(\xi^e)) = \hat{\phi}_1(\xi^e)$$
$$\phi_2^e(x(\xi^e)) = \hat{\phi}_2(\xi^e)$$

Therefore

$$\left.\frac{\partial \phi_i^e(x)}{\partial x}\right|_{x(\xi^e)} = \frac{\partial \phi_i^e(x(\xi^e))}{\partial \xi^e}\frac{\partial \xi^e}{\partial x} = \frac{\partial \hat{\phi}_i(\xi^e)}{\partial \xi^e}\frac{1}{J^e} = B_i(\xi^e)\frac{2}{h^e}$$

The integral over the domain can be split into a sum of integrals over each element, and since the only nonzero shape functions over an element are its own, we can say

$$K_{ij} = \int_0^L \frac{\partial \phi_i}{\partial x}\kappa(x)\frac{\partial \phi_j}{\partial x}dx$$

$$K_{ij}^e = \int_{x_e}^{x_{e+1}} \frac{\partial \phi_i^e}{\partial x}\kappa(x)\frac{\partial \phi_j^e}{\partial x}dx$$

$$\mathbf{K} = \underset{e=1}{\overset{n}{A}} \mathbf{K^e}$$

Where A is the special matrix summation specified in tutorial 3 of the course. To finish writing the integral form of the stiffness matrix for a master element we simplify to

$$K_{ij}^e = \int_{-1}^{1} \frac{1}{J^e}B_i\kappa(x(\xi^e))\frac{1}{J^e}B_j J^e d\xi^e = \frac{2}{h^e}\int_{-1}^{1} B_i\kappa(x(\xi^e))B_j d\xi^e$$

It is easy to see from the expression for $\kappa(x)$ that each of its values perfectly covers one element at a time. Therefore using already calculated matrices we can say

$$K_{ij}^e = \frac{2\kappa_e}{h^e}\int_{-1}^{1} B_i B_j d\xi^e$$

$$\Rightarrow \mathbf{K^e} = \frac{\kappa_e}{h^e}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} = 4\kappa_e\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

We can then directly calculate $\mathbf{K}$.

$$\mathbf{K} = 4\begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 3 & -2 & 0 & 0 \\ 0 & -2 & 5 & -3 & 0 \\ 0 & 0 & -3 & 5 & -2 \\ 0 & 0 & 0 & -2 & 2 \end{bmatrix}$$

Similarly, using calculated matrices,

$$\mathbf{M} = \underset{e=1}{\overset{n}{A}} \mathbf{M^e}$$

$$\mathbf{M^e} = \frac{h^e}{6}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = \frac{1}{24}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

$$\Rightarrow \mathbf{M} = \frac{1}{24}\begin{bmatrix} 2 & 1 & 0 & 0 & 0 \\ 1 & 4 & 1 & 0 & 0 \\ 0 & 1 & 4 & 1 & 0 \\ 0 & 0 & 1 & 4 & 1 \\ 0 & 0 & 0 & 1 & 2 \end{bmatrix}$$

It should be reminded that our weak formulation of the problem is only correct $\forall 1 < i < 5$. The boundary conditions provide information about the behavior of the system at $i = 1$ and $i = 5$:

$$\phi_j(0)a_j(t) = 0 \Rightarrow a_1(t) = 0$$

$$\phi_j(1)a_j(t) = 1 \Rightarrow a_5(t) = 1$$

$$\Rightarrow \dot{a}_1(t) = \dot{a}_5(t) = 0$$

Therefore to convert the weak formulation to matrix notation, we should apply these conditions to the inner rows of the mass and stiffness matrices:

$$\frac{1}{24}\begin{bmatrix}4 & 1 & 0\\ 1 & 4 & 1\\ 0 & 1 & 4\end{bmatrix}\dot{\mathbf{a}} + 4\begin{bmatrix}3 & -2 & 0\\ -2 & 5 & -3\\ 0 & -3 & 5\end{bmatrix}\mathbf{a} = \begin{bmatrix}0\\ 0\\ 8\end{bmatrix}$$

To differentiate between matrices and vectors which refer only to inner nodes as opposed to the original complete node set with the boundary conditions, we will use the *mathcal* font. As such the above matrices will be denoted $\mathcal{M}$, $\mathcal{K}$, and $\mathcal{F}$ respectively.

1.2

To formulate a general time-marching scheme we simply apply the general trapezoidal rule to our weak formulation. This was derived in tutorials and since in our case $\mathcal{F}$ is constant, can write the resulting scheme as

$$\breve{\mathcal{K}}a^{t+\Delta t} = \bar{\mathcal{K}}a^{t} + \bar{\mathcal{F}}$$

Where

$$\breve{\mathcal{K}} = \mathcal{M} + \alpha\Delta t\mathcal{K}$$

$$\bar{\mathcal{K}} = \mathcal{M} - \Delta t(1-\alpha)\mathcal{K}$$

$$\bar{\mathcal{F}} = \Delta t\mathcal{F}$$

1.3

We are instructed to obtain numerical stability, which can be checked by confirming that the spectral radius of the state transition matrix is less than 1. This is only a concern for the explicit scheme, being that it is conditionally stable. Therefore we check the following:

$$\alpha = 0 \Rightarrow \left(\rho(\breve{\mathcal{K}}^{-1}\bar{\mathcal{K}}) < 1 \iff \rho(\mathcal{M}^{-1}(\mathcal{M} - \Delta t\mathcal{K})) < 1 \iff \rho(\mathbf{I} - \Delta t\mathcal{M}^{-1}\mathcal{K}) < 1\right)$$

To find the eigenvalues analytically we would need to solve an overly complicated quintic equation so we will simply use MATLAB and iteratively find that $\Delta t_{crit} \in [0.0065, 0066]$.
We would now like to solve the first two time steps, so to be safe we choose $\Delta t = 0.0065$. For $\alpha = 0$:

$$\mathbf{a}^{\Delta t} = \begin{bmatrix}0\\ \breve{\mathcal{K}}^{-1}(\bar{\mathcal{K}}a^{0} + \bar{\mathcal{F}})\\ 1\end{bmatrix} = \begin{bmatrix}0\\ 0.0223\\ -0.0891\\ 0.3343\\ 1\end{bmatrix};\ \mathbf{a}^{2\Delta t} = \begin{bmatrix}0\\ \breve{\mathcal{K}}^{-1}(\bar{\mathcal{K}}a^{\Delta t} + \bar{\mathcal{F}})\\ 1\end{bmatrix} = \begin{bmatrix}0\\ -0.0846\\ 0.1853\\ 0.2752\\ 1\end{bmatrix}$$

For $\alpha = 0.5$:

$$\mathbf{a}^{\Delta t} = \begin{bmatrix}0\\ 0.0002\\ -0.0026\\ 0.2245\\ 1\end{bmatrix};\ \mathbf{a}^{2\Delta t} = \begin{bmatrix}0\\ -0.0064\\ 0.0738\\ 0.3212\\ 1\end{bmatrix}$$

For $\alpha = 1$:

$$\mathbf{a}^{\Delta t} = \begin{bmatrix}0\\ 0.0009\\ 0.0218\\ 0.1780\\ 1\end{bmatrix};\ \mathbf{a}^{2\Delta t} = \begin{bmatrix}0\\ 0.0074\\ 0.0728\\ 0.2872\\ 1\end{bmatrix}$$

1.4

See MATLAB code.

1.5

The steady state problem is a simple static system.

$$\mathbf{a}^{t\to\infty} \approx \begin{bmatrix} 0 \\ 0.4286 \\ 0.6429 \\ 0.7857 \\ 1 \end{bmatrix}$$

1.6

For $n = 50$ we choose $\Delta t = 1 \cdot 10^{-5}$ which we can do because the spectral radius of the state transition matrix is less than 1. For the other two stable schemes we use the previous $\Delta t = 0.006$. See appendix for figures.

1.7

Once again we can check the spectral radius of the state transition matrix and we find that $\Delta t_{crit} = (2.5 \pm 0.5) \cdot 10^{-5}$. To display a diverging numerical we use $\Delta t = 3 \cdot 10^{-5}$. We can see from figure 4 that the darker blue lines indicating later timesteps are oscillating more and more, which is indicative of a diverging solution.

1.8

From this section onward the solution to the PDE will be denoted as U and not T so that the latter may represent time-dependent functions. To find the first three eigenvalues of the problem we return to the given PDE and separate variables, introducing an approximate from of the solution $\tilde{U}(x,t) = T(t) \cdot X(x)$. We can then apply the Galerkin method to the spatial factor of the solution approximating it as $\tilde{X} = \phi_j a_j$, and solve the resulting eigenvalue problem. This process has been done in the tutorial for a more general heat equation—the only difference in our case being that κ is a function of space. This changes nothing in the derivation and manifests only in the general expression for the stiffness matrix. The result in our case is

$$\mathbf{K}^{-1}\mathbf{M}\mathbf{a} = \frac{1}{\gamma_n}\mathbf{a}$$

Where $\gamma_n \in \mathbb{R}$ represents the n eigenmodes of the system. As mentioned, the expression for the stiffness matrix ends up identical to the one derived for the time-marching scheme despite the presence of $\kappa(x)$, and the mass matrix expression is always the same. Therefore, using linear elements results in identical local mass and stiffness matrices as derived for the domain discretization using n elements in the time-marching scheme.
It should be mentioned that the weak formulation once again is only valid for the inner nodes of the system. We assemble the matrices using $n = 100$ elements, correct the system by reducing the matrices to refer only to inner node values using the new boundary conditions, and calculate the first three (largest) eigenvalues:

$$\lambda_1 = 0.0648 \Rightarrow \gamma_1 = 15.4349 \Rightarrow \omega_1 =$$

$$\lambda_1 = 0.0139 \Rightarrow \gamma_1 = 71.9484$$

$$\lambda_1 = 0.0061 \Rightarrow \gamma_1 = 164.7758$$

Corresponding modes are found in figure 5.

1.9

Looking at figure 6, we observe the correct rate of convergence of 2 for linear elements for the first eigenvalue error.

2

2.1

Before applying the Galerkin method we define the residuals in the domain and on the non-Dirichlet boundaries, introducing an approximate form of the solution $\tilde{T} = \phi_j a_j$.:

$$R = -\kappa\nabla^2\tilde{T} = -\kappa\nabla^2\phi_j a_j$$

$$r_L = \left.\frac{d\tilde{T}}{dn}\right|_{\Gamma_L} = \left.\frac{d\phi_j}{dn}a_j\right|_{\Gamma_L}$$

$$r_R = \left.\frac{d\tilde{T}}{dn}\right|_{\Gamma_R} = \left.\frac{d\phi_j}{dn}a_j\right|_{\Gamma_R}$$

Where Γ_R, Γ_L, Γ_T, and Γ_B denote the right, left, top, and bottom boundaries of the problem respectively, and $\Gamma_R + \Gamma_L + \Gamma_T + \Gamma_B$. Now we demand

$$\int_\Omega \phi_i\big(-\kappa\nabla^2\phi_j a_j\big)d\Omega + \int_{\Gamma_L}\phi_i\left(\frac{d\phi_j}{dn}a_j\right)_{\Gamma_L} ds + \int_{\Gamma_R}\phi_i\left(\frac{d\phi_j}{dn}a_j\right)_{\Gamma_R} ds = 0$$

Using

$$\int_\Omega \phi_i[-\nabla\cdot(\kappa\nabla\phi_j)]d\Omega = \kappa\int_\Omega \nabla\phi_i\cdot\nabla\phi_j d\Omega - \kappa\int_\Gamma \phi_i(\nabla\phi_j\cdot\mathbf{n})ds$$

and

$$(\nabla\phi_j\cdot\mathbf{n}) = \frac{d\phi_j}{dn}$$

We can simplify to

$$\left(\kappa\int_\Omega \nabla\phi_i\cdot\nabla\phi_j d\Omega\right)a_j - \left(\int_{\Gamma_T}\phi_i(\nabla\phi_j\cdot\mathbf{n})ds\right)a_j - \left(\int_{\Gamma_B}\phi_i(\nabla\phi_j\cdot\mathbf{n})ds\right)a_j = 0$$

We see that the Neumann term in the mixed boundary condition has vanished which is the expected result. The remaining two line integrals should also be discounted but more mathematical tools are needed to explain how, so this is addressed later. This is the weak formulation of the problem and we can write it as

$$(\kappa\mathbf{K} - \mathbf{T} - \mathbf{B})\mathbf{a} = 0$$

2.2

To solve the problem with n bilinear quadrilateral elements we introduce N nodal points (x_j, y_j), and the following transformation $\forall e \leq n \in \mathbb{N}$ (e is not in Einstein notation) along with basis functions as derived in tutorials:

$$\mathbf{v}(\xi^e,\eta^e) = \begin{bmatrix} x(\xi^e,\eta^e) & y(\xi^e,\eta^e)\end{bmatrix} = \hat{\boldsymbol{\phi}}(\xi^e,\eta^e)\mathbf{V^e}$$

Where

$$\mathbf{V^e} = \begin{bmatrix} x_1^e & y_1^e \\ x_2^e & y_2^e \\ x_3^e & y_3^e \\ x_4^e & y_4^e \end{bmatrix}$$

And the four shape functions per element are defined inside the element by

$$\boldsymbol{\phi^e}(x,y) = \hat{\boldsymbol{\phi}}(\xi^e(x,y),\eta^e(x,y))$$

$$\hat{\boldsymbol{\phi}}(\xi,\eta) = \frac{1}{4}\begin{bmatrix}(1-\eta)(1-\xi) & (1-\eta)(1+\xi) & (1+\eta)(1+\xi) & (1+\eta)(1-\xi)\end{bmatrix}$$

Outside the element, its shape functions vanish. The relation between (x_j^e, y_j^e), ϕ_i^e and (x_j, y_j), ϕ_i (node-numbering) will be defined manually once a number of elements for solution is specified.

It should be noted that from this point the ∇ operator will be defined as a column vector, thus performing a column-wise gradient on row vectors according to whatever coordinates within which the operand is defined. Using this, for convenience we will define

$$\hat{\mathbf{B}}(\xi,\eta) = \nabla\hat{\boldsymbol{\phi}}(\xi,\eta) = \frac{1}{4}\begin{bmatrix} -(1-\eta) & (1-\eta) & (1+\eta) & -(1+\eta) \\ -(1-\xi) & -(1+\xi) & (1+\xi) & (1-\xi) \end{bmatrix}$$

The N integrals over the domain Ω can each be split into a sum of integrals over the domain of each element Ω^e, and since the only nonzero shape functions inside an element are its own, we can say

$$\mathbf{K} = \int_{\Omega} (\nabla\boldsymbol{\phi})^T \cdot \nabla\boldsymbol{\phi} d\Omega$$

$$\mathbf{K^e} = \int_{\Omega^e} (\nabla\boldsymbol{\phi^e})^T \cdot \nabla\boldsymbol{\phi^e} d\Omega$$

The combination of all $\mathbf{K^e}$'s to form $\mathbf{K}$ is done manually by assigning a node to each row of the global matrix. We use $\mathbf{K^e}$ to simplify the math using the above transformation in the following way:

$$\mathbf{K^e} = \int_{-1}^{1}\int_{-1}^{1} (\nabla(\boldsymbol{\phi^e}(x,y))\big|_{\mathbf{v}(\xi^e,\eta^e)})^T \cdot (\nabla(\boldsymbol{\phi^e}(x,y))\big|_{\mathbf{v}(\xi^e,\eta^e)}) |\mathbf{J^e}| d\xi^e d\eta^e$$

Where

$$\mathbf{J^e} = \nabla\mathbf{v}(\xi^e,\eta^e) = \begin{bmatrix} \frac{\partial x(\xi^e,\eta^e)}{\partial\xi^e} & \frac{\partial y(\xi^e,\eta^e)}{\partial\xi^e} \\ \frac{\partial x(\xi^e,\eta^e)}{\partial\eta^e} & \frac{\partial y(\xi^e,\eta^e)}{\partial\eta^e} \end{bmatrix} = \nabla(\hat{\boldsymbol{\phi}}(\xi^e,\eta^e))\mathbf{V^e} = \hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e}$$

Then we note that

$$\nabla(\boldsymbol{\phi^e}(x,y))\big|_{\mathbf{v}(\xi^e,\eta^e)} = \nabla\begin{bmatrix}\xi^e(x,y) & \eta^e(x,y)\end{bmatrix} \nabla(\boldsymbol{\phi^e}(x(\xi^e,\eta^e), y(\xi^e,\eta^e))) = (\mathbf{J^e})^{-1}\nabla(\hat{\boldsymbol{\phi}}(\xi^e,\eta^e)) = (\mathbf{J^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e)$$

For example, for row 1 corresponding to x:

$$\frac{\partial\phi_i^e(x,y)}{\partial x}\bigg|_{x(\xi^e,\eta^e),y(\xi^e,\eta^e)} = \frac{\partial\phi_i^e(x(\xi^e,\eta^e),y(\xi^e,\eta^e))}{\partial\xi^e}\frac{\partial\xi^e}{\partial x} + \frac{\partial\phi_i^e(x(\xi^e,\eta^e),y(\xi^e,\eta^e))}{\partial\eta^e}\frac{\partial\eta^e}{\partial x}$$

$$= \frac{\partial\hat{\phi}_i(\xi^e,\eta^e)}{\partial\xi^e}\frac{\partial\xi^e}{\partial x} + \frac{\partial\hat{\phi}_i(\xi^e,\eta^e)}{\partial\eta^e}\frac{\partial\eta^e}{\partial x}$$

Therefore

$$\mathbf{K^e} = \int_{-1}^{1}\int_{-1}^{1} ((\mathbf{J^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e))^T \cdot ((\mathbf{J^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e)) |(\mathbf{J^e})| d\xi^e d\eta^e$$

$$\Rightarrow \mathbf{K^e} = \int_{-1}^{1}\int_{-1}^{1} ((\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e))^T \cdot ((\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e)) |(\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})| d\xi^e d\eta^e$$

At this point we can also point out that $0 < y_i < 1 \Rightarrow \phi_i(x,y_i) = 0\ \forall i$. This allows us to get rid of the top and bottom line integrals $\forall i : 0 < y_i < 1$. This means that the weak formulation now looks like

$$\kappa\mathbf{K}\mathbf{a} = 0$$

As such, in future matrix equations it should be noted that rows corresponding to nodes for which this condition does not hold are technically incorrect and are to be replaced by the Dirichlet boundary condition at these nodes:

$$y_i = 1 \Rightarrow a_i = 0\ \forall i;\ y_i = 0 \Rightarrow a_i = 30\ \forall i$$

2.3

We would now like to calculate the above matrix explicitly using Gaussian quadrature twice. Since our integrand is a linear function with respect to both variables individually we only need $2p-1 \geq 1 \Rightarrow \mathbb{N} \ni p \geq 1$ Gauss point for each quadrature and our integral value will be exact.

$$\xi_k^e = 0;\ w_k = 2$$

$$\int_{-1}^{1}((\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e))^T\cdot((\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(\xi^e,\eta^e))|(\hat{\mathbf{B}}(\xi^e,\eta^e)\mathbf{V^e})|d\xi^e$$

$$=2((\hat{\mathbf{B}}(0,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(0,\eta^e))^T\cdot((\hat{\mathbf{B}}(0,\eta^e)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(0,\eta^e))|(\hat{\mathbf{B}}(0,\eta^e)\mathbf{V^e})|$$

$$\eta_k^e=0;\ w_k=2$$

$$\mathbf{K^e}=4((\hat{\mathbf{B}}(0,0)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(0,0))^T\cdot((\hat{\mathbf{B}}(0,0)\mathbf{V^e})^{-1}\hat{\mathbf{B}}(0,\eta^e))|(\hat{\mathbf{B}}(0,0)\mathbf{V^e})|$$

Where

$$\hat{\mathbf{B}}(0,0)=\frac{1}{4}\begin{bmatrix}-1 & 1 & 1 & -1\\ -1 & -1 & 1 & 1\end{bmatrix}$$

2.4

We would now like to solve the problem using $n=1$ element. We number the nodes counterclockwise starting at the bottom left node to ensure a bijective mapping, which means:

$$\mathbf{V^e}=\begin{bmatrix}0 & 0\\ 2 & 0\\ 2 & 1\\ 0 & 1\end{bmatrix}$$

$$\Rightarrow\mathbf{K}=\mathbf{K^e}=\frac{1}{32}\begin{bmatrix}5 & 3 & -5 & -3\\ 3 & 5 & -3 & -5\\ -5 & -3 & 5 & 3\\ -3 & -5 & 3 & 5\end{bmatrix}$$

As mentioned, the resulting system is not correct until boundary conditions are accounted for:

$$\begin{bmatrix}1 & 0 & 0 & 0\\ 0 & 1 & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{bmatrix}\mathbf{a}=\begin{bmatrix}30\\ 30\\ 0\\ 0\end{bmatrix}$$

As we can see, since in this case all nodes were on Dirichlet boundaries, the approximate solution is trivial.

$$\Rightarrow\mathbf{a}=\begin{bmatrix}30\\ 30\\ 0\\ 0\end{bmatrix};\ \boldsymbol{\phi}=\boldsymbol{\phi^e}$$

$$\Rightarrow\tilde{T}=\boldsymbol{\phi}\mathbf{a}$$

2.5

We would now like to write the global stiffness matrix using $n=2$ square elements. The only way to have two square elemets is for them to be side by side. We once again number nodes counterclockwise, starting at the bottom left node, defining $\mathbf{V}$ as the global node list for clarity:

$$\mathbf{V}=\begin{bmatrix}0 & 0\\ 1 & 0\\ 2 & 0\\ 2 & 1\\ 1 & 1\\ 0 & 1\end{bmatrix}$$

Element 1 will be the left element with nodes 1, 2, 5 and 6, and element 2 will be the right element with nodes 2, 3, 4, and 5.

$$\mathbf{V^1}=\begin{bmatrix}0 & 0\\ 1 & 0\\ 1 & 1\\ 0 & 1\end{bmatrix}$$

$$\mathbf{V^2} = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 2 & 1 \\ 1 & 1 \end{bmatrix}$$

We can now calculate what the global stiffness matrix would look like before correcting for Dirichlet conditions.

$$\mathbf{K^1} = \frac{1}{8}\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{K^2} = \frac{1}{8}\begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{bmatrix}$$

$$\Rightarrow \mathbf{K} = \frac{1}{8}\begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 2 & 0 & -1 & 0 & -1 \\ 0 & 0 & 1 & 0 & -1 & 0 \\ 0 & -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & -1 & 0 & 2 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Appendix

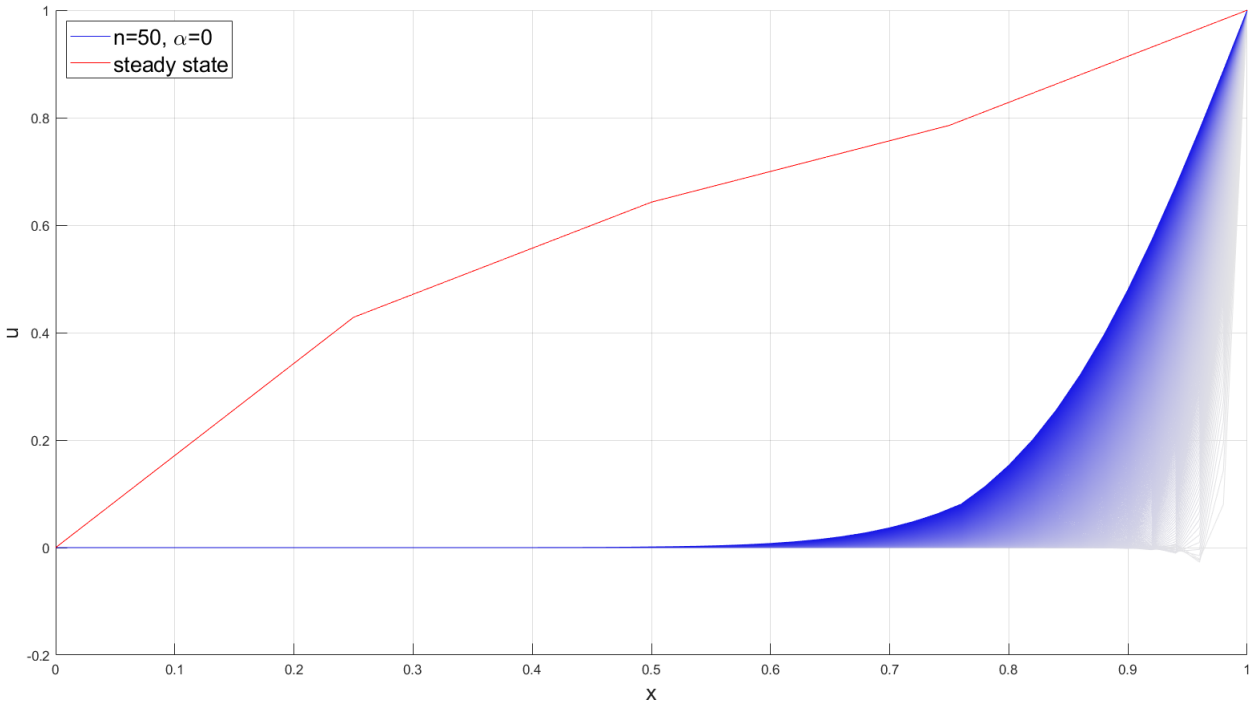

Figure 1: Q1.6: Numerical Steady State Solution vs. Explicit Scheme Solution Using 50 Elements and 500 iterations

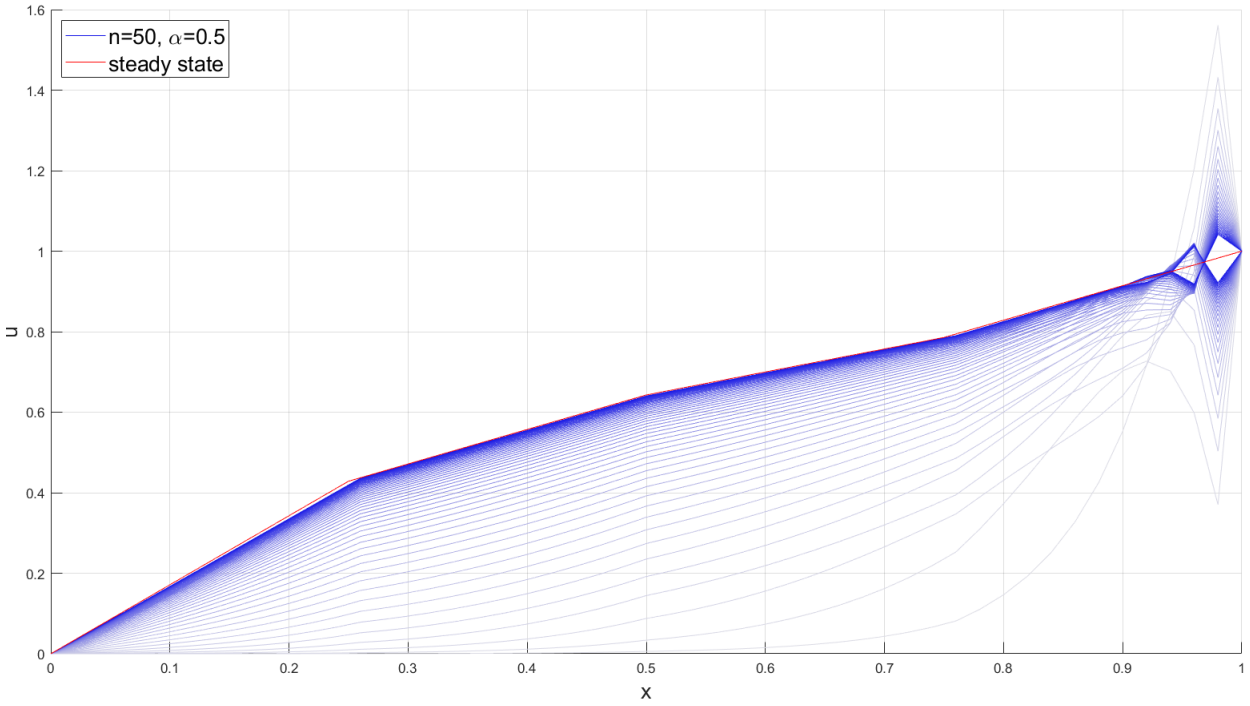

Figure 2: Q1.6: Numerical Steady State Solution vs. Midpoint Scheme Solution Using 50 Elements and 50 iterations

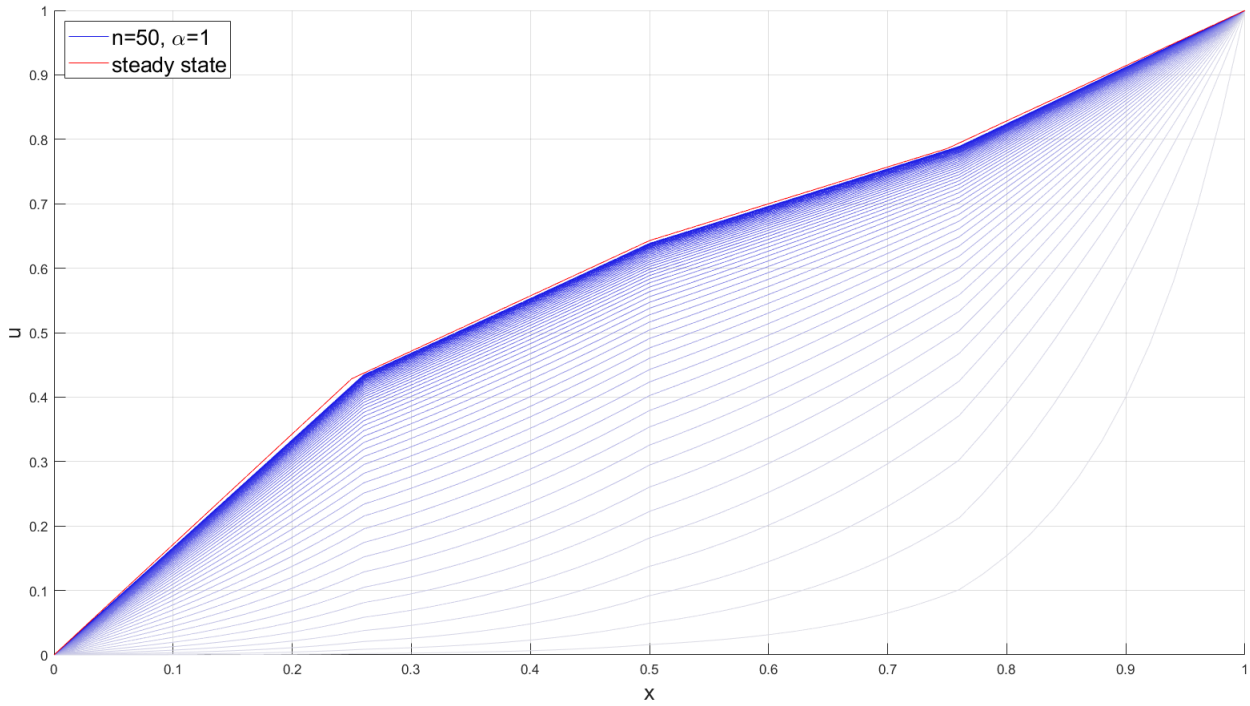

Figure 3: Q1.6: Numerical Steady State Solution vs. Implicit Scheme Solution Using 50 Elements and 50 iterations

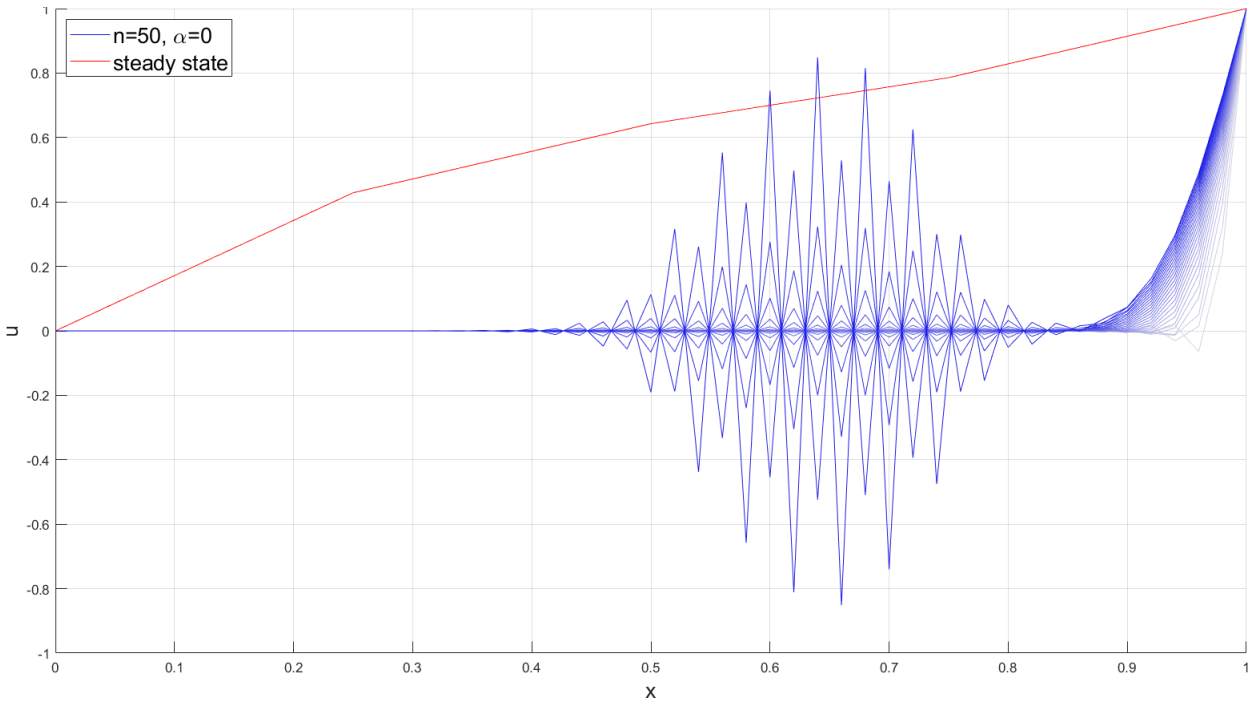

Figure 4: Q1.7: Numerical Steady State Solution vs. Explicit Scheme Diverging Solution Using 50 Elements and 27 iterations

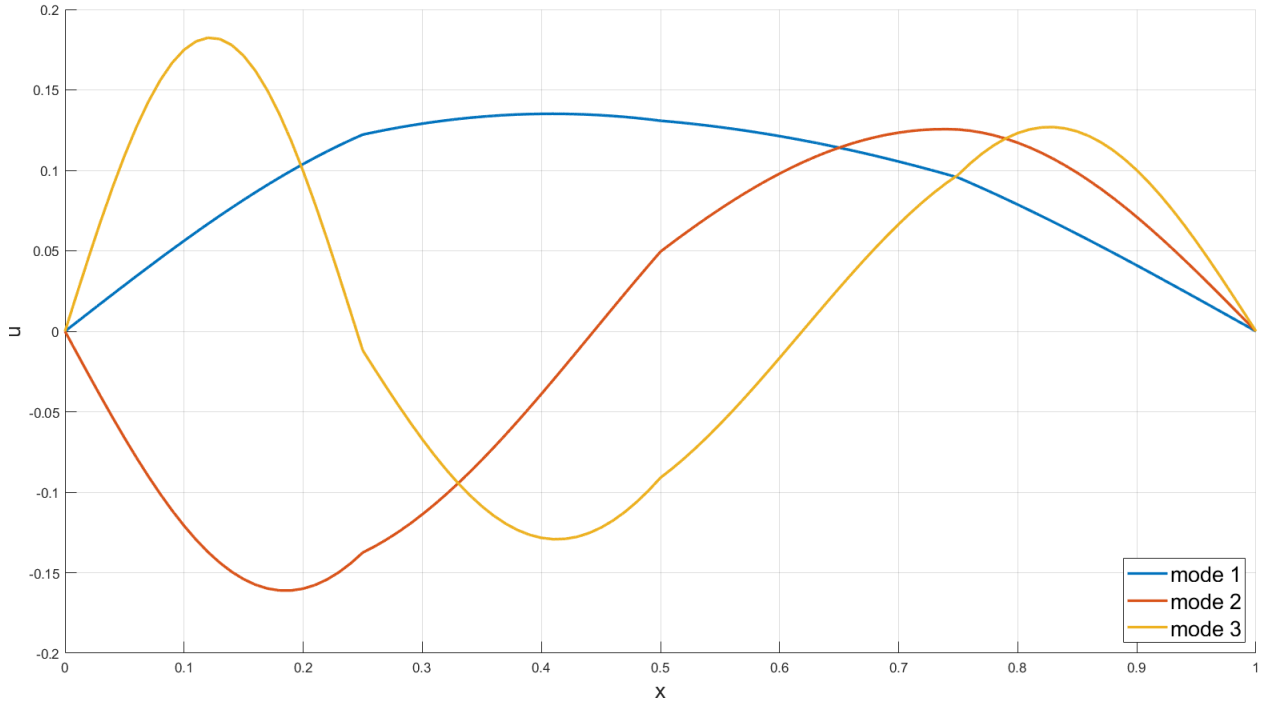

Figure 5: Q1.8: Largest three modes of solution using 100 elements

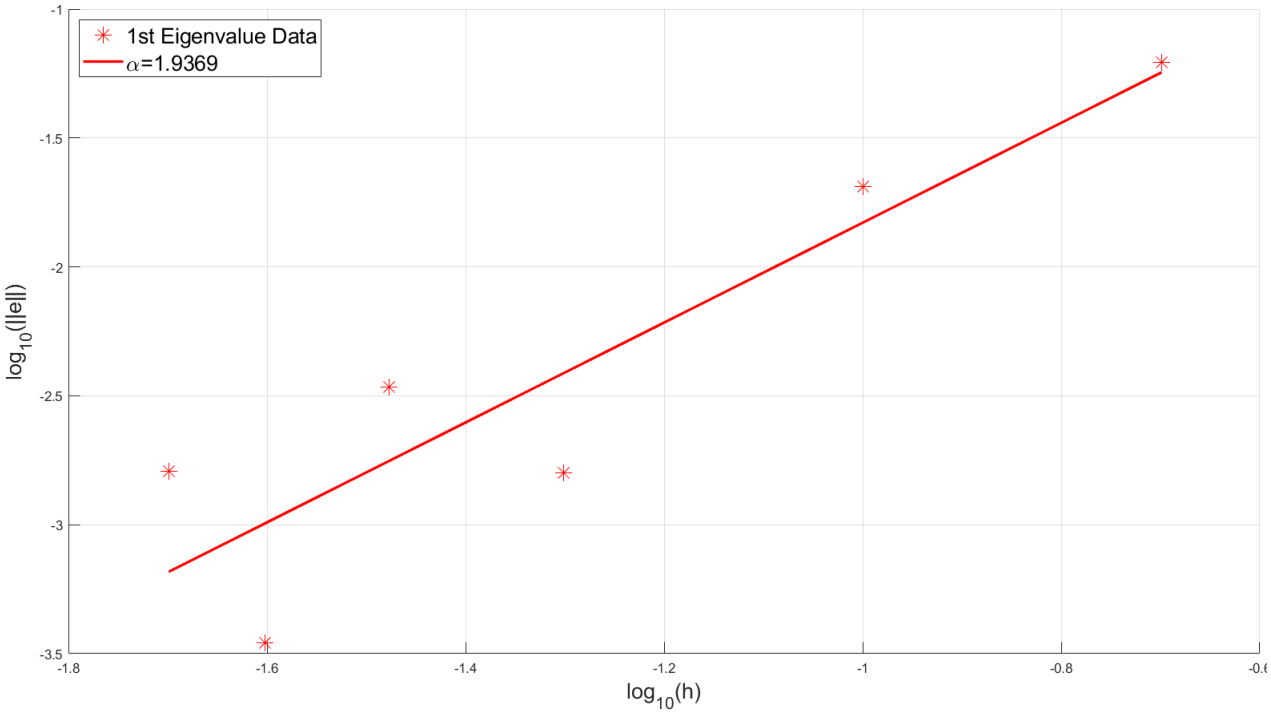

Figure 6: Q1.9: Logarithmic Error of First Eigenvalue vs. Logarithmic Element Size h